

On the Role of Hadamard Gates in Quantum Circuits

D. J. Shepherd¹

Received October 25, 2005; accepted March 23, 2006; Published online May 27, 2006

We study a reduced quantum circuit computation paradigm in which the only allowable gates either permute the computational basis states or else apply a “global Hadamard operation”, i.e. apply a Hadamard operation to every qubit simultaneously. In this model, we discuss complexity bounds (lower-bounding the number of global Hadamard operations) for common quantum algorithms: we illustrate upper bounds for Shor’s Algorithm, and prove lower bounds for Grover’s Algorithm. We also use our formalism to display a gate that is neither quantum-universal nor classically simulable, on the assumption that Integer Factoring is not in BPP.

KEY WORDS: Quantum depth in quantum circuits; Fourier Hierarchy; Shor’s algorithm with Toffoli gates only; lower-bounding Grover’s algorithm.

PACS: 03.67. Lx; 03.67. Ta.

1. INTRODUCTION

A Quantum Circuit (or Quantum Logic Network) is usually presented as being composed both of *wires* that carry qubits and *gates* that tap those wires to modify the qubits they carry.⁽⁵⁾ In Sec. 2, we specify the notation used to describe computation with quantum circuits, and specify exactly which features we shall be allowing within the kinds of circuits we wish to consider.

The main focus is to enquire about the difference it makes if we limit to using ‘classical’ gates (ones which preserve the set of computational basis states) and ‘global Hadamard transforms’ (where a Hadamard gate is applied once to each qubit). We will show in Sec. 3 that our imposed limitations do not limit computational power in any real sense,

¹Department of Computer Science, University of Bristol, Bristol, B58 1TH, UK. E-mail: shepherd@compsci.bristol.ac.uk

and in subsequent sections will discuss what algorithms and algorithm-primitives tend to look like within this model. This model is closely related to the complexity class Fourier Hierarchy, defined in Ref. 9. The motivation comes not from physical considerations pertinent to the task of fabricating a quantum information processor, but from the desire to analyse a fairly natural-looking measure of circuit complexity that is not apparent within the standard model, *viz* the number of these global Hadamard transformations needed. The reason for requiring that the Hadamard operations be applied on every qubit is that we are not necessarily thinking of *actively* applying them, so much as just *passively* changing what is meant by the computational basis, and therefore, changing the interpretation of future gates or measurements. More will be said on this in Sec. 4.4.

Section 4 will discuss the idea of replacing the Fourier Transform in Shor's Algorithm with the kind of transform that is simplest within our limited model, and look at how this affects the gate-complexity of the algorithm. The concept of *Order Finding*, on which Shor's algorithm is based, turns out to be a very natural primitive within the present context.

In Sec. 5, we extend a result of Ref. 7 to show the trade-offs in different complexity measures relevant to Grover's algorithm. This is undertaken in a similar spirit to the work of Ref. 8, showing why Grover's algorithm is essentially non-parallelisable.

2. NOTATION AND TERMINOLOGY

A circuit incorporates a finite number of wires, which run right through the circuit, not terminating prematurely (i.e. no 'adaptive' measurements). The *width* of a circuit is taken to be the number of such qubit wires used.

The input to the circuit is taken to be a (classical description of the) state in which the wires are initialised, usually a simple computational basis state.

Each wire, i.e. qubit, codes quantum data with respect to a tensor component $\mathcal{C}$ within the full space $\mathcal{C}^n$ that is associated with the totality of n qubits, and these component spaces are given a *computational basis* (Z) and a *Hadamard basis* (X), both of which are orthonormal:

$$\begin{aligned} Z\text{-basis} &:= \{|0\rangle, |1\rangle\}, \\ X\text{-basis} &:= \left\{ \begin{aligned} |+\rangle &:= H|0\rangle := \frac{|0\rangle+|1\rangle}{\sqrt{2}}, \\ |-\rangle &:= H|1\rangle := \frac{|0\rangle-|1\rangle}{\sqrt{2}} \end{aligned} \right\}. \end{aligned} \quad (1)$$

The data resident on a circuit may equally well be described by a trace-1 Hermitian density operator, which will be of rank 1 if the state is pure. We will use an algebra of Pauli symbols

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \quad (2)$$

to describe such density operators explicitly, where necessary. Subscripts on these symbols will distinguish between different qubits. The same Pauli symbols may be used to describe unitary transforms (conjugations of the density operator by unitary actions), so for example the *C-Not gate* may be written

$$\Lambda_c(X_t) = \frac{1}{2}(1 + Z_c + X_t - Z_c \cdot X_t), \quad (3)$$

where c and t label the *control* and *target* qubits, respectively. More generally, the *generalised Toffoli gate* may be written

$$\Lambda_C(X_T) = 1 - \left(1 - \prod_{t \in T} X_t\right) \prod_{c \in C} \left(\frac{1 - Z_c}{2}\right). \quad (4)$$

In classical terms, these gates flip each of the target bits if each of the control bits is set to 1, and otherwise acts as the identity. The two sets of qubits C and T must, of course, be disjoint, and T must not be empty. (To see that (4) is unitary, observe that it is the identity minus twice a product of commuting projectors, and hence geometrically a reflection.) We call the number of such gates used the *size* of the circuit (other authors give similar—though different—definitions of *size*, but this point will not be important in this discussion).

As well as allowing for Generalised Toffoli gates (to be used an arbitrary number of times on arbitrary qubits within a circuit), we also allow for a Hadamard map to be applied simultaneously to every qubit in the circuit:

$$H^\infty = \prod_k \left(\frac{X_k + Z_k}{\sqrt{2}} \right), \quad (5)$$

where the product is taken over every index, without exception. We use the expression *quantum-depth* to count the number of such operations used within a circuit.

For the purposes of obtaining computational outcomes, projective single-qubit measurements in the computational basis may be applied to the quantum output of a circuit. We will use the term *output* to refer to the

sublist of the measured qubits that carry the data in which we are interested, since in many circuit designs it happens that many wires do not output useful data.

To avoid unnecessary discussion at the bit-level, we will use the term *register* to denote a list of wires, and *output register* to denote those wires that carry useful data at the end of the circuit. If a circuit has been designed to be useful with different inputs, then the term *input register* will be used to refer to those wires that are initialised differently on different runs of the circuit, and the term *ancilla register* will refer to those wires that are always initialised in the same way for each run. If a circuit is *not* designed to be used with different inputs, then the term *ancilla register* is used instead to refer to the qubits that are not in the *output register*.

3. UNIVERSALITY

It is not too hard to see that there are constant-width and -depth circuits, which can be used to implement *local* Hadamard transforms within this model, with certain ancillæ used as catalysts. For example, the circuit

$$\Lambda_{cd}(X_b) \cdot H^\infty \cdot \Lambda_{cd}(X_a) \cdot H^\infty \cdot \Lambda_{cd}(X_b) \quad (6)$$

clearly has effects which are local to qubits a, b, c, d . In particular, it maps states as follows:

$$\begin{aligned} |1-00\rangle &\mapsto |1-++\rangle, \\ |1-01\rangle &\mapsto |1--+\rangle, \\ |1-10\rangle &\mapsto |1-+-\rangle, \\ |1-11\rangle &\mapsto |1---\rangle. \end{aligned} \quad (7)$$

Therefore, we could take qubits a, b, c to be an ancilla in the state $|1-?\rangle$ (the question-mark denoting a totally arbitrary qubit), and use the circuit above, followed or preceded by a swap on qubits c and d , to achieve a *local* Hadamard transform on qubit d while returning the other qubits to their former states. (Strictly speaking, the data on qubit c is not returned to its former state unless that state were fully mixed. But if we never ‘care’ about that qubit, then no problem arises.) Notice also that the *swap gate* can be constructed from three *C-Not* gates, which are amongst the Generalised Toffolis. (It is interesting that an ancilla was necessary for this construction.)

We should point out that although Toffoli gates and local Hadamard gates are insufficient for achieving all unitary transforms directly, they are

well known to constitute a *Universal Set* of computing gates,⁽²⁾ and so the limitations that we have imposed should not be thought of as especially restrictive. Yet one does indeed seem to end up with some limitations on computing power if it is not possible to initialise ancilla qubits with both X - and Z -basis elements.

4. CIRCUITS WITH SMALL QUANTUM-DEPTH

4.1. A Modification to Shor's Algorithm

In this section, we will show how Shor's algorithm may, for all practical purposes, be divided into two parts; the first of which performs some *order-finding* operations, the second of which converts 'order-finding data' into an actual problem answer (and is called *postprocessing*.) We will show that each of these parts *on their own* can be performed efficiently without using any Hadamard maps, though Hadamard maps are necessary within the interface. (While the division of Shor's algorithm into two parts is not in itself a novel concept, it is hoped that a detailed analysis of its implementation within this particular model has something useful to teach about its algorithmic complexity.)

4.2. Order Finding

For *Order-Finding*, which is the first part of our algorithm, let there be an ancilla register whose contents will code for elements of some presentation of a cyclic group $G = \langle g \rangle$, when in the computational Z -basis, and which more generally codes for superpositions of group elements. Let there be an input register that starts out with each qubit in the pure state $|+\rangle$.

Let U be the map on the ancilla register that carries $|x\rangle$ to $|x \cdot g\rangle$, which (by hypothesis, and by universality of Toffoli gates for classical computation) can be constructed from a reasonable number of (generalised) Toffoli gates. We should not generally care how U maps vectors that are not in the span of the coding for G (which necessarily exist whenever $|G|$ is not a power of 2), so to keep matters simple we shall require that U acts as the identity on computational basis vectors that do not code for elements of G . Again, it is straightforward to find efficient Toffoli circuitry for doing this by simply taking classical circuitry for the same problem and making it reversible.

Kitaev made the observation in Ref. 4 that simple circuitry will assist in the estimation of one of the eigenvalues of such a map, U . Ignoring the

eigenspace of eigenvalue 1, the remaining eigenspaces of the domain of U will each be 1-dimensional, and of the form

$$|\lambda_\omega\rangle := |G|^{-1/2} \sum_{j=1}^{|G|} \omega^{-j} |g^j\rangle \quad (8)$$

for ω a complex $|G|$ th root of unity.

Then, if the ancilla register were to hold such an eigenvector, we could apply the gate $\Lambda_r(U_{\text{anc}})$ —a gate likewise constructible from generalised Toffoli maps—which implements U on the ancilla controlled on the Z-basis setting of an input-register qubit r ; and thus effect the transformation

$$|+\rangle_r = \frac{|0\rangle_r + |1\rangle_r}{\sqrt{2}} \mapsto \frac{|0\rangle_r + \omega|1\rangle_r}{\sqrt{2}} = \frac{(1+\omega)}{2}|+\rangle_r + \frac{(1-\omega)}{2}|-\rangle_r. \quad (9)$$

The classical bit issued from the measurement (in the Hadamard basis) of the register wire would therefore, be biased as $[\cos^2(\theta) : \sin^2(\theta)]$, where $\omega =: \exp(2i\theta)$. If we would rather not make a Hadamard-basis measurement for classical postprocessing but would instead process the data within the quantum circuit, then we may of course simply transfer the data back into the computational basis with a H^∞ operation, instead of measuring it.

The aim in any rendition of Shor's algorithm is not to consider such data in isolation but to chain together several of these $\Lambda_r(U_{\text{anc}})$ gates, using a different register wire r for each. Note that these gates all commute, so there is no need to prespecify the order in which they are to be performed. In this manner, many bits are provided for assisting in the estimation of θ .

As it stands, this algorithm does not amount to much, because the classical bits issued by such a circuit are highly correlated and carry little information collectively. Indeed, one would obtain a remarkably poor estimate of θ if this approach alone were relied upon. [It would require exponentially many samples of the $(\cos^2(\theta) : \sin^2(\theta))$ probability distribution to estimate ω to a sufficient accuracy.] The real key to Kitaev's observation is to use other unitary maps that have the same eigenspaces. These are the maps of the form $\Lambda_r(U_{\text{anc}}^c)$, and they all commute. By chaining these maps into the computation, data can be collected not just on some specific ω eigenvalue, but on its powers, ω^c , also. This process can be extended to include as many commuting maps as we have patience and spare wires for, provided that it is no more difficult to find circuitry for implementing $\Lambda_r(U_{\text{anc}}^c)$.

4.3. Ancilla in Order-Finding

In practice, it will not be possible to load the ancilla register with a non-trivial eigenvector. Instead, we load it with an arbitrary state. Provided that this initial ancilla state does not overlap much with the eigenvalue-1 eigenspace, the effect of the Order-Finding part of the algorithm just described will be to generate within the input-register data that is in a superposition of states, which (when understood in the Hadamard basis) estimate the different possible θ values. To a great extent, it makes no difference whether the ancilla superposition is coherent or incoherent, so either an arbitrary pure state or a mixed state could be used. Even a fully mixed state can be used for the ancilla, since the eigenvalue-1 eigenspace does not dominate the ‘useful’ spaces superpolynomially.

To make this more explicit, we show how to determine a stochastically chosen θ in the case where it is easy to find simple circuits for exponentially high powers of a certain U . Starting with $a \cdot b$ input wires, each initialised as before to $|+\rangle$; for $j \in \{1, 2, \dots, a\}$, for c_j being an exponentially growing series of positive integers, we can reserve b wires for each value of j , each for controlling one implementation of U^{c_j} . We are free to choose any values we like for the c_j , but not all choices will be appropriate for completing the task of eigenvalue estimation. More will be said about the c_j values later.

$$\begin{aligned} \sum_{\omega} \alpha_{\omega} |\lambda_{\omega}\rangle_{\text{anc}} \otimes |+\rangle_{\text{input}}^{\otimes ab} &\mapsto \\ \sum_{\omega} \alpha_{\omega} |\lambda_{\omega}\rangle_{\text{anc}} \otimes \bigotimes_{j=1}^a \left(\frac{(1 + \omega^{c_j})|+\rangle + (1 - \omega^{c_j})|-\rangle}{2} \right)^{\otimes b}. \end{aligned} \quad (10)$$

The traditional rendering of Shor’s algorithm proceeds by applying an appropriate Quantum Fourier Transform (QFT) to the input register at this stage,⁽⁵⁾ but since the QFT is not readily implementable using small numbers of the gates that we are wishing to consider, we shall instead consider this output state (10) to conclude the *Order-Finding* part of our algorithm.

4.4. Quantum Depth

Sometimes it is convenient to think of the full algorithm as having quantum-depth of 1, allowing $|+\rangle$ states as input and where the two parts (order-finding and postprocessing) are separated by a H^{∞} operator. By performing a H^{∞} on the output state of (10) it is clear that the data in

which we are interested are transformed from the X -basis to the Z -basis, where they can be processed with Toffoli gates.

Sometimes it is better to think of our algorithm as having quantum-depth 0, where the postprocessing part is performed not inside the circuit but on a classical computer *after* the quantum algorithm has ended and the circuit output has been measured in the X -basis (assuming X -basis measurement is available).

Sometimes it makes more sense to think of our algorithm as having quantum-depth 2, for then we are free to use the canonical $|0\rangle$ input on each wire, and require no measurement in the X -basis at all.

Therefore, rather than trying to answer definitively the question, ‘What is the quantum-depth of our implementation of Shor’s algorithm?’ we shall instead proceed simply by describing how classical processing of samples from

$$\{x_j \sim \text{Binomial}(b, \sin^2(\theta \cdot c_j))\}_{j=1}^a \quad (11)$$

can be used to provide a good enough estimate of ω , where ω is chosen at random with probability $|\alpha_\omega|^2$ (see expression (10)) and $\omega = \exp(2i\theta)$. [To be clear, x_j will be an integer between 0 and b , most likely around about $b \cdot \sin^2(\theta \cdot c_j)$, & $c \dots$]

4.5. Parallelisation of Order Finding

Before proceeding to explain the (classical) computational tasks involved in processing the output of *Order Finding*, we make a few comments about the circuitry used so far. Toffoli circuitry is used (as explained in Sec 4.2) to implement

$$\prod_j \Lambda_{R_j}(U_{\text{anc}}^{c_j}), \quad (12)$$

where each R_j is a set of b wires used to control the application of the gate U^{c_j} , which is just a modular-multiplication mapping. It is well known (e.g. see Ref. 6 for the case of integer multiplication) that circuitry for this kind of functionality can frequently be constructed to have overall *poly-log* depth in the input length, with a merely *polynomial* overhead in the circuit width, paid for by addition of certain extra ancilla gates that are to be initialised to $|0\rangle$. To provide such ancillæ within our model would add a small constant factor to the overall quantum-depth of the algorithm, as well as increasing its width; but if we are prepared to pay such a cost, then the circuitry of *Order Finding* can be said to be ‘exponentially parallelisable’.

4.6. Eigenvalue Estimation

Since having rejected the QFT as a way of recovering the desired (classical-description) approximation to ω , we must instead address the *purely classical* problem of figuring out how to choose the parameters a , b , and c_j , and how to use the samples x_j in order to form the approximation to ω in an efficient manner. This is not an altogether straightforward task, because it depends on the problem being solved, e.g. one chooses parameters slightly differently from the integer factorisation problem than for the discrete logarithm problem.

A little experimentation in the general case, implementing a sampling strategy for random θ in expression (11) on a (classical!) computer, has led us to the conclusion that the optimal choice for c_j is *not* consistently 2^j as used with the QFT method (see Ref. 5). This is because in the case where the binary expansion of θ has a long run of zeroes or ones after j places, then there is remarkably little data in x_j .

Experimentally, we found that if θ needs to be estimated to within one part in K , then it suffices (for a reasonable chance of estimating θ correctly) to take $b = 3 \log \log(K)$ and to let the c_j take every value between 1 and K that either has just one bit set or else has two bits set which are not more than $b/2$ places apart. Using those c_j values with *two* bits set enables one to write code which can successfully ‘bridge over’ those region of the Real line (for θ) that include numbers whose binary expansions have early long runs of zeroes or ones. The precise details of this technique are omitted for brevity, but the problem is not technically difficult. Proving rigorous limits in this regime however, remains an open problem.

In the case, where θ is known *a priori* to be a multiple of $2\pi/\phi$ for some unknown integer $\phi \sim \sqrt{K}$, then this resolution suffices (with constant probability) to recover θ exactly, by the method of continued fractions. There is extensive literature on applying eigenvalue estimation to specific problem-instances.

4.7. Generalising ‘Quantum-Depth = 2’

We saw in the prequel that (a version of) Shor’s algorithm may be implemented in our computational model with overall quantum-depth of 2, starting from a trivial Z -basis state and ending with a Z -basis measurement. Consider now a ‘general’ algorithm of quantum-depth 2, and the computational path it follows. Starting from Z -basis state $|a\rangle$ (where $a \neq 0$) on n wires; apply a H^∞ map; then a Z -basis-permutation T ; then another H^∞ ; then measure the result $|c\rangle$ in the Z -basis. This computation, and the resulting probability distribution, may be denoted by

$$\begin{aligned}
|a\rangle &\mapsto 2^{-n} \sum_{b,c} (-1)^{\langle a,b \rangle} (-1)^{\langle T(b),c \rangle} |c\rangle \\
&\mapsto \left\{ \text{Prob}(c) = 2^{-n} \sum_b (-1)^{\langle a,b \rangle} (-1)^{\langle T(b),c \rangle} \right\} \\
&= \left\{ \begin{array}{l} \text{Prob}(0) = 0 \\ \text{Prob}(c) = 2^{1-n} \cdot \sum_{b:\langle a,b \rangle=0} (-1)^{\langle T(b),c \rangle} \end{array} \right\}. \quad (13)
\end{aligned}$$

It is somewhat surprising to reflect that this little formula contains virtually all the power needed to factorise large integers. Note that as well as being implementable in the model this paper has described, algorithms with this low-quantum-depth are also effectively implementable (modulo classical postprocessing) on a quantum computer that has access *only* to gates of the form $H^\infty \cdot \Lambda_C(X_T) \cdot H^\infty$, together with Z-basis inputs and outputs. (Note that despite our notation, this gate acts trivially on all but a constant number of qubits, and so it is ‘small’ in the usual sense.) While such a machine could help us factorise large integers, it would be of little use for ordinary ‘classically easy’ tasks. The example of this ‘conjugated Toffoli gate’ essentially resolves one of the open problems (problem 4) listed in Ref. 1: The gate is neither classically simulable nor universal for quantum computation in any reasonable sense. This assertion is justified by the observations : (1) if it were classically simulable, then *Order Finding* would be classically simulable too, and hence *Factorisation* would be in BPP; (2) if it were universal for BQP then a polynomial usage of the conjugate-Toffoli could approximate one usage of an ordinary Toffoli gate, so by symmetry a polynomial usage of the Toffoli gate could approximate one conjugate-Toffoli gate, and hence the latter would be classically simulable. To find uses for such a ‘conjugate-Toffoli machine,’ beyond applications of Shor’s algorithm, we leave as an open problem.

5. GROVER’S ALGORITHM

5.1. Simple Oracle for the Unstructured Search Problem

Define a *simple Grover Oracle* G_x , with *hidden data* x , to be an application of the following one-register phase-flip map. (Note that the Grover Oracle does not return a description of a circuit for implementing that mapping, rather it simply returns an application of that mapping, as per the ‘black-box’ model.)

$$G_x|z\rangle := (-1)^{\{x=f(z)\}}|z\rangle. \quad (14)$$

Here f is taken to be a fixed function acting on the labels of the computational basis of the register, and $\{x = f(z)\}$ is 1 if $x = f(z)$ and zero otherwise. We let N denote the cardinality of the range of f . It is intended that f have very little complexity: typically it will just check the first $\log(N)$ qubits of the register.

A simple Grover Oracle for an unstructured *search problem* may be simulated in the model pertinent to our present study, provided that there is an efficient way of determining classically whether $x = f(z)$ for any given z . For example, this can be managed with a two-qubit ancilla in the state $|1-\rangle$, so that no matter how many times H^∞ has been applied, there will still be a qubit in the state $|-\rangle$ on which to target a generalised Toffoli gate, so as to implement the necessary sign-flip that the oracle (simulation) requires.

5.2. Search Algorithm Notation

A *Grover Search Algorithm* expressed in our model comprises a circuit of $\Lambda_C(X_T)$ gates and H^∞ gates and an input, and a computational basis measurement at the output which, with high probability, will result in a $|z\rangle$ that indicates the hidden data $x = f(z)$. We are interested in lower-bounding not the depth of the circuit but rather its quantum-depth and the number of oracle calls used. The notation for the algorithm is

$$G_x^{k_T} \cdot H^\infty \dots H^\infty \cdot G_x^{k_2} \cdot H^\infty \cdot G_x^{k_1} \cdot |\psi\rangle,$$

where

$$G_x^{k_t} := T_{t,k_t} \cdot G_x \cdots T_{t,2} \cdot G_x \cdot T_{t,1} \cdot G_x \cdot T_{t,0}, \quad (15)$$

where $T_{t,j}$ are predetermined permutations of the Z basis. The quantum-depth of this circuit is $T - 1$ and the number of oracle calls is $\sum_{t=1}^T k_t$.

It would be trivial to specify how to implement Grover's algorithm if we did not care about quantum-depth; simply interleave calls to the oracle with maps of the form $H^\infty \cdot T_0 \cdot H^\infty$, for some appropriate T_0 . But the questions we should like to ask concern lower-bounds and trade-offs between the quantum-depth and the number of oracle calls required.

5.3. Categories of Success

To be precise, we must state for which kinds of algorithm are we asking for bounds. Consider deterministic algorithms, algorithms that have worst-case probability above some bound, and algorithms that have average-case probability above some bound.

Deterministic algorithms are not the focus of this study. While of theoretic interest, such algorithms are less amenable to the kinds of analysis that we wish to deploy, and the model described in this article probably has little to contribute to their study.

Algorithms with *bounded worst-case probability* are those for which, no matter what the hidden data turns out to be, the probability of the algorithm succeeding exceeds $1 - \epsilon$ for some constant ϵ . Since the success probability of an algorithm can always be ‘amplified’ by running it several times (either in series or in parallel), without loss of generality we always take ϵ to be less than $1/2$. This kind of algorithm is probably the easiest to bound (see Ref. 3 for a good general technique for non-geometric proofs in this area), but they shall not be of primary concern to us here.

Algorithms with *bounded average-case probability* are the most relevant for an algorithm designer, since they allow for the possibility of breaking some symmetry in the problem in such a way that certain answers will be substantially easier to recover. They are algorithms for which *a priori* the success probability exceeds $1 - \epsilon$.

For highly symmetric problems (such as *Unstructured Search*) one generally expects such algorithms to be the same as those with bounded worst-case probability, but the techniques for proving average-case bounds can sometimes be different. Given many instances of a certain kind of ‘naturally arising’ problem, an algorithm with bounded average-case probability will be most welcome, since it will almost certainly serve to address many of those instances. It is less use if the instances in hand are actually derived from a different kind of problem that just happens to lie ‘nearby’ in terms of complexity, since there might be no reason to assume that what is ‘average’ for one class of problem will translate into average instances as derived from that class.

Henceforth, we will consider only lower bounds for the resource requirements of algorithms with bounded average-case probability, since a lower bound for resource requirements in the average case implies a lower bound for resource requirements in the worst case, but the converse does not follow.

5.4. Lower Bounds

In this section, we give some preliminary lemmas leading to a proof of the following:

Theorem 1. Any Grover Search Algorithm, as defined in Sec. 5.2 with the notation of Eq. (15), with some constant lower bound for its average-case expected probability of success, will require sufficient oracle

calls between H^∞ stages so that

$$\sum_{t=1}^T \sqrt{k_t} = \Omega(\sqrt{N}) \quad (16)$$

holds true asymptotically.

Note that this means an algorithm might make one oracle call in each of $\sim \sqrt{N}$ quantum-depth-phases, or it might make $\sim N$ calls without any quantum-depth, or it might interpolate these extremes.

Lemma 1. Let $w_{x,t}$ be entries of a 2-dimensional array of non-negative real numbers. Let $R_t = \sqrt{\sum_x w_{x,t}^2}$ be the 2-norm of the rows, and let $C_x = \sum_t w_{x,t}$ be the 1-norm of the columns. Then let $R = \sum_t R_t$ be the 1-norm of the R s, and let $C = \sqrt{\sum_x C_x^2}$ be the 2-norm of the C s. Then $R \geq C$.

To prove this, let t be the vector whose x th entry is $w_{x,t}$. Then using the Euclidean inner product, we can rewrite $R_t = \sqrt{\langle t, t \rangle}$. By expanding the expression for C^2 we get $C^2 = \sum_{s,t} \langle s, t \rangle$. Likewise, $R^2 = (\sum_t \sqrt{\langle t, t \rangle})^2 = \sum_{s,t} \sqrt{\langle s, s \rangle \cdot \langle t, t \rangle}$. So it suffices to show that always

$$\sqrt{\langle s, s \rangle \cdot \langle t, t \rangle} \geq \langle s, t \rangle \quad (17)$$

but this is a basic (Euclidean) trigonometric inequality. $\square$

Lemma 2. Given three unit vectors in a Euclidean Geometry, the sum of the absolute values of the sines of any two inter-angles is no less than the absolute value of the sine of the third inter-angle.

This is trivial in two dimensions. The general problem is three-dimensional. Consider three unit vectors whose components are $(1, 0, 0)$, $(a, b, 0)$ and (c, d, e) , with $|b| \geq 0$. (No generality is lost with this choice.) Then it only remains to show that

$$b + \sqrt{1 - c^2} \geq \sqrt{1 - (ac + bd)^2}. \quad (18)$$

We begin by squaring both sides, so that we are required to show that

$$b^2 + 1 - c^2 + 2b\sqrt{1 - c^2} \geq 1 - (ac + bd)^2. \quad (19)$$

Next, we try to balance between d and e . The case $d=0$ immediately satisfies (19). The case $e=0$ reduces (19) to the requirement $|ac + bd| \leq 1$, which is a basic trigonometric result, as the reader can easily check. The only remaining case occurs when the partial derivative of (19) w.r.t. d vanishes, i.e. when $(ac + bd)b = 0$. The $b=0$ sub-case is easily dismissed, and the other sub-case is defined by $ac = -bd$. We can use this to reduce (19) to $b^2 - c^2 + 2b\sqrt{1-c^2} \geq 0$, which by solving for b is identical with the condition $b \geq 1 - \sqrt{1-c^2}$ on the range of validity. We can also use it to obtain $(1-b^2)c^2 = b^2d^2$, and hence $|c| \leq b$. And this latter inequality guarantees the condition sought : $\sqrt{1-c^2} \geq \sqrt{1-b^2} \geq 1-b$. ■

Lemma 3. For (t_x) a list of real numbers,

$$\sum_{x=1}^N t_x^2 \leq Np \Rightarrow \sum_{x=1}^N t_x \leq N\sqrt{p}. \quad (20)$$

The variance of t_x is non-negative, that is

$$\frac{1}{N} \sum_x t_x^2 - \left(\frac{1}{N} \sum_x t_x \right)^2 \geq 0 \quad (21)$$

and the lemma follows directly from this observation. ■

To prove Theorem 1, we proceed as follows. Referring back to the notation of Sec. 5.2, let

$$\begin{aligned} U_t &:= H^\infty \cdot T_{t,k_t} \cdots T_{t,0}, \\ |\psi_t\rangle &:= U_t \cdot U_{t-1} \cdots U_1 \cdot |\psi\rangle, \\ P_{x,t} &:= U_t^\dagger \cdot H^\infty \cdot G_x^{k_t}, \\ |\psi_{x,v,t}\rangle &:= U_v \cdot P_{x,v} \cdot U_{v-1} \cdots P_{x,v-1} \cdots U_{t+1} \cdot P_{x,t+1} \cdot |\psi_t\rangle, \end{aligned} \quad (22)$$

and write

$$\begin{aligned} \text{path}_{x,t}(z) &:= \{f \cdot T_{t,0}(z) = x\} \oplus \cdots \oplus \{f \cdot T_{t,k_t-1} \cdots T_{t,0}(z) = x\}, \\ W_{x,t}(|\phi\rangle) &:= \sum_z |\langle \phi | z \rangle|^2 \{\text{path}_{x,t}(z) \equiv 1\} \quad (\leq 1). \end{aligned} \quad (23)$$

Then it follows that

$$P_{x,t}|z\rangle = (-1)^{\text{path}_{x,t}(z)}|z\rangle \quad (24)$$

and states defined in (22) may be compared with each other according to

$$\begin{aligned}\langle \psi_{x,T,t} | \psi_{x,T,t-1} \rangle &= \langle \psi_{t-1} | \cdot P_{x,t} \cdot | \psi_{t-1} \rangle \\ &= 1 - 2W_{x,t}(|\psi_{t-1}\rangle), \\ 1 - |\langle \psi_{x,T,t} | \psi_{x,T,t-1} \rangle|^2 &\leq 4W_{x,t}(|\psi_{t-1}\rangle).\end{aligned}\quad (25)$$

Square-rooting expression (25), summing that over the t values, and further approximating by many applications of Lemma 2, then squaring, we obtain

$$1 - |\langle \psi_T | \psi_{x,T,0} \rangle|^2 \leq 4 \left(\sum_{t=1}^T \sqrt{W_{x,t}(|\psi_{t-1}\rangle)} \right)^2. \quad (26)$$

Using the trigonometric notation

$$\cos(|\psi_T\rangle, |\psi_{x,T,0}\rangle) = |\langle \psi_T | \psi_{x,T,0} \rangle| \quad (27)$$

and summing expression (26) over the x values, we obtain,

$$\sum_x \sin^2(|\psi_T\rangle, |\psi_{x,T,0}\rangle) \leq 4 \sum_x \left(\sum_{t=1}^T \sqrt{W_{x,t}(|\psi_{t-1}\rangle)} \right)^2. \quad (28)$$

Denote by F the left side of this inequality. Apply Lemma 1 to the right side of this inequality after square-rooting, and it yields

$$\sqrt{F} \leq 2 \sum_{t=1}^T \sqrt{\sum_x W_{x,t}(|\psi_{t-1}\rangle)}. \quad (29)$$

Now, by Eq. (23), we know that

$$\begin{aligned}\sum_x W_{x,t}(|\psi_{t-1}\rangle) &= \sum_z |\langle \psi_{t-1} | z \rangle|^2 \sum_x \{\text{path}_{x,t}(z) \equiv 1\} \\ &\leq \sum_z |\langle \psi_{t-1} | z \rangle|^2 \cdot k_t \\ &= k_t.\end{aligned}\quad (30)$$

So if we put these observations together (substitute (30) into (29)), we see

$$F \leq 4 \left(\sum_{t=1}^T \sqrt{k_t} \right)^2. \quad (31)$$

It suffices to show that this F is lower-bounded by $\Omega(N)$ whenever the algorithm has expected success-probability $\Theta(1)$. Let $|C_x\rangle$ denote the closest unit vector to $|\psi_{x,T,0}\rangle$ that is in the span of $\{|z\rangle : f(z)=x\}$. Since the output of the algorithm is $H^\infty \cdot |\psi_{x,T,0}\rangle$, we can use a measurement in the basis given by $H^\infty|C_x\rangle$, and by the Born rule establish that the expected probability of success is

$$p = N^{-1} \sum_x \cos^2(|\psi_{x,T,0}\rangle, |C_x\rangle), \quad (32)$$

which we rewrite as

$$\sum_x \sin^2(|\psi_{x,T,0}\rangle, |C_x\rangle) = N(1-p). \quad (33)$$

It follows from Lemma 3 that

$$\sum_x |\sin(|\psi_{x,T,0}\rangle, |C_x\rangle)| \leq N\sqrt{1-p}. \quad (34)$$

And since the $|C_x\rangle$ are all mutually orthogonal, $\sum_x \cos^2(|\psi_T\rangle, |C_x\rangle) \leq 1$, which we write as

$$\sum_x \sin^2(|\psi_T\rangle, |C_x\rangle) \geq N-1. \quad (35)$$

Now apply Lemma 3 directly to the definition of F (LHS of inequality (28)) to obtain

$$\sum_x |\sin(|\psi_T\rangle, |\psi_{x,T,0}\rangle)| \leq \sqrt{N \cdot F}. \quad (36)$$

Putting together lines (34)–(36), and applying Lemma 2, we obtain

$$\sqrt{N \cdot F} + N\sqrt{1-p} \geq N-1, \quad (37)$$

whence $p = \Theta(1) \Rightarrow F = \Omega(N)$, as required.

6. CONCLUSIONS

We have introduced a new theoretical model for quantum circuits, designed to highlight one aspect of the way in which quantum computation differs from classical computation. We have thence illustrated a little of what can be achieved within limited quantum-depth, by analysis of the

two main (or well-known) algorithms of quantum information processing; showing that ‘hard’ classical problems can sometimes be ‘solved quantumly’ using only Toffoli gates, and using the model to add to the growing literature on ‘algorithmic trade-offs.’ We have developed a few tools for facilitating these analyses, and exemplified quantum a gate that is probably neither BQP universal nor classically simulable. Further exploration of the power of computing within very small (e.g. constant) quantum depth remains an interesting issue for future research.

ACKNOWLEDGMENTS

Thanks are due to Richard Jozsa for much proof-reading and helpful discussions. This work has been sponsored by CESG.

REFERENCES

1. S. Aaronson and D. Gottesman, *Phys. Rev. A* **70**, 052328 (2004).
2. D. Aharonov, *A Simple Proof that Toffoli and Hadamard are Quantum Universal* (quant-ph/0301040).
3. A. Ambainis, *J. Comp. Syst. Sci.* **64**, (2002).
4. A. Y. Kitaev, (quant-ph/9511026).
5. M. Nielsen and I. Chuang, *Quantum Computation and Quantum Information* (Cambridge University Press, Cambridge, 2000).
6. C. Papadimitriou, *Computational Complexity*, Ch. 15 (Addison Wesley, New York, 1994).
7. Y. Shi, *Quantum and Classical Tradeoffs* (quant-ph/0312213) To appear in Theoretical Computer Science (2005).
8. C. Zalka, *Phys. Rev. A* **60**, 2746–2751 (1999).
9. http://qwiki.caltech.edu/wiki/Complexity_Zoo.